Unit 3: Knowledge Representation and Probabilistic Reasoning

SubUnit 3.4: Semantic Networks and Frames

Kiran Bagale

IOE, Thapathali Campus

December 24, 2025

Structured Knowledge Representation

Beyond Logic

While logic is powerful, humans often think in terms of **objects**, **relationships**, and **hierarchies**.

Structured Representations:

- More **intuitive** and **visual**
- Capture **taxonomic** relationships naturally
- Support **inheritance** and **default reasoning**
- Bridge human cognition and machine representation

Two Major Approaches:

1. **Semantic Networks**: Graph-based representation
2. **Frames**: Structured object-oriented representation

Reference: Rich et al. (2009), Chapter 5; Russell & Norvig (2020), Section 8.5

Semantic Networks: Overview

Definition

A **semantic network** is a directed graph where:

- **Nodes** represent concepts, objects, or events
- **Edges** represent relationships between nodes

Historical Context:

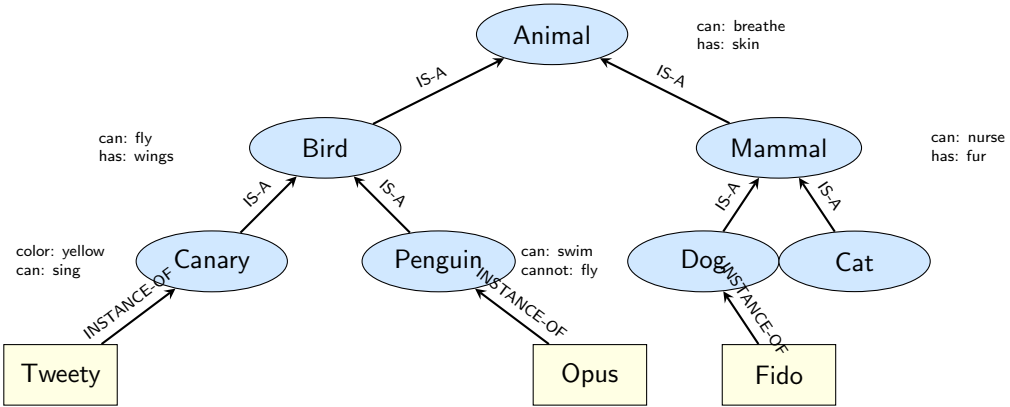
- Developed in the 1960s-70s (Quillian, Collins & Loftus)
- Based on theories of human memory organization
- Used in early AI systems and natural language processing

Common Relations:

- **IS-A**: Taxonomic hierarchy (Bird IS-A Animal)
- **HAS-A/PART-OF**: Component relationships (Car HAS-A Engine)
- **INSTANCE-OF**: Instance to class (Tweety INSTANCE-OF Bird)
- Other domain-specific relations (OWNS, LOCATED-IN, etc.)

Reference: Rich et al. (2009), Section 5.2

Semantic Network: Example



Classic example adapted from Rich et al. (2009)

Property Inheritance in Semantic Networks

Key Concept: Properties are inherited down the IS-A hierarchy

Inheritance Example

Query: "Can Tweety breathe?"

Reasoning:

1. Tweety INSTANCE-OF Canary
2. Canary IS-A Bird
3. Bird IS-A Animal
4. Animal has property "can breathe"
5. Therefore: Tweety can breathe

Advantages:

- **Economy**: Store properties once at highest level
- **Efficiency**: Avoid redundant storage
- **Maintainability**: Update property once, affects all descendants

Handling Exceptions in Semantic Networks

Problem: Not all instances follow the general rule

The Penguin Problem

- Birds can fly (general rule)
- Opus is a penguin
- Penguins are birds
- But penguins **cannot** fly! (exception)

Solutions:

1. **Exception Nodes**: Store exceptions explicitly at lower levels
   - Penguin node has property "cannot: fly"
2. **Non-monotonic Inheritance**: Closer properties override inherited ones
   - Search from specific to general
   - First match wins
3. **Default Logic**: Mark properties as defaults vs. strict rules

Semantic Networks: Strengths and Weaknesses

Advantages:

- **Intuitive**: Visual and easy to understand
- **Natural hierarchies**: Captures taxonomies well
- **Efficient storage**: Through inheritance
- **Cognitive plausibility**: Models human memory
- **Graph algorithms**: Can use standard graph traversal

Limitations:

- **Informal semantics**: Meaning not always clear
- **Limited expressiveness**: Hard to represent complex logic
- **No standard inference**: Different implementations vary
- **Exception handling**: Can be ad-hoc
- **Ambiguity**: Multiple interpretations possible

Key Issue

Semantic networks lack formal semantics—what exactly does an edge mean? This led to development of more formal approaches.

Reference: Russell & Norvig (2020), Section 8.5.1

Frames: Structured Representations

Definition

A **frame** is a data structure for representing stereotypical situations or objects, consisting of:

- **Slots**: Attributes or properties
- **Fillers**: Values for those attributes
- **Facets**: Constraints, defaults, procedures

Historical Background:

- Proposed by Marvin Minsky (1975)
- Influenced by human perception and memory studies
- Foundation for object-oriented programming concepts

Key Idea:

- Represent knowledge as **structured objects**
- Support **inheritance** through frame hierarchies
- Enable **procedural attachment** (if-needed, if-added)

Frame Structure: Detailed Example

Frame: Person

PERSON
- IS-A: Mammal
- SLOT: Name
  - TYPE: String
  - DEFAULT: "Unknown"
- SLOT: Age
  - TYPE: Integer, RANGE: [0, 150]
  - IF-NEEDED: Calculate from birthdate
- SLOT: Height
  - TYPE: Float, UNIT: cm
- SLOT: Birthdate
  - TYPE: Date
- SLOT: Parents
  - TYPE: List of PERSON
  - CARDINALITY: 2

Slot Components:

- **Value**: Actual data
- **Type**: Data type constraint
- **Default**: Value if unspecified
- **Range**: Valid value range
- **Cardinality**: Number of values
- **IF-NEEDED**: Procedure to compute value
- **IF-ADDED**: Procedure when value added
- **IF-REMOVED**: Procedure when value removed

Benefits:

- Rich constraint specification
- Active knowledge (procedures)
- Validation support

Frame Hierarchies and Inheritance

Frame Hierarchy

ANIMAL
SLOT: Can-Breathe: Yes
SLOT: Habitat: ?

BIRD (IS-A: ANIMAL)
SLOT: Can-Fly: Yes
SLOT: Has-Wings: Yes

PENGUIN (IS-A: BIRD)
SLOT: Can-Fly: No *(override)*
SLOT: Can-Swim: Yes
SLOT: Habitat: Antarctica

Tweety (INSTANCE-OF: BIRD)
SLOT: Name: "Tweety"
SLOT: Color: Yellow

Reference: Rich et al. (2009), Section 5.3.2

Inheritance Rules:

1. Values propagate **downward**
2. More specific values **override** general ones
3. Missing values can be **inherited**
4. **Multiple inheritance** possible

Example Query:

- Can Tweety fly?
  $\rightarrow$ Yes (inherited from BIRD)
- Can Opus (a penguin) fly?
  $\rightarrow$ No (overridden at PENGUIN)
- What is Tweety's habitat?
  $\rightarrow$ Unknown (not specified)

Procedural Attachment in Frames

Active Knowledge: Frames can contain procedures, not just data

Three Types of Procedures:

1. **IF-NEEDED (Demons)**:
   - Executed when slot value is accessed but empty
   - Used for computed values
   - Example: Calculate age from birthdate
2. **IF-ADDED (Triggers)**:
   - Executed when new value is added to slot
   - Used for constraint checking or updates
   - Example: When age is set, update "adult" status
3. **IF-REMOVED**:
   - Executed when value is removed
   - Used for maintaining consistency

Example

Frame: Employee

SLOT: Salary → IF-ADDED: Update tax bracket, notify payroll system

SLOT: Age → IF-NEEDED: Calculate from (current-year - birth-year)

Comparison: Semantic Networks vs. Frames

Aspect	Semantic Networks	Frames
Structure	Graph (nodes + edges)	Structured objects (slots + facets)
Visualization	Highly visual, intuitive	More textual, detailed
Properties	Attached to nodes	Organized in slots with facets
Procedures	Generally not supported	Rich procedural attachment
Constraints	Limited	Extensive (type, range, cardinality)
Inheritance	Simple IS-A links	Complex with defaults and overrides
Best for	Taxonomies, simple hierarchies	Complex objects, OOP-like systems
Formalism	Less formal	More structured and formal

Relationship

Frames can be viewed as a **more structured** version of semantic networks with richer slot descriptions and procedural capabilities.

Applications and Modern Perspectives

Historical Applications:

- **Expert Systems**: Medical diagnosis (MYCIN), planning systems
- **Natural Language Processing**: Understanding and generation
- **Computer Vision**: Scene interpretation and object recognition
- **Database Systems**: Conceptual modeling

Modern Descendants:

- **Object-Oriented Programming**: Classes, inheritance, methods
- **Description Logics**: Formal semantic networks (basis for OWL)
- **Knowledge Graphs**: Google, Amazon, Facebook (billions of entities)
- **Ontologies**: Medical ontologies (SNOMED), WordNet
- **Linked Data**: Semantic Web technologies (RDF, RDFS, OWL)

Key Insight

While pure semantic networks and frames are less common today, their principles underlie modern knowledge representation systems and object-oriented design.

Summary

Key Takeaways:

Semantic Networks:

- Graph-based, visual, intuitive representation
- Natural for hierarchies and taxonomies (IS-A, HAS-A)
- Support property inheritance
- Limited formal semantics

Frames:

- Structured object representation with slots and facets
- Rich constraint specification (types, defaults, ranges)
- Support procedural attachment (IF-NEEDED, IF-ADDED)
- Foundation for object-oriented programming

Both approaches:

- Bridge human cognition and machine representation
- Enable efficient knowledge organization through inheritance
- Influenced modern KR systems (knowledge graphs, ontologies, OOP)